POISONED DATASET

Who changed the labels?

A two-player game of data poisoning and scarce audits

2 PLAYERS · TARGET 15–20 MINUTES · DRAFT 0.1

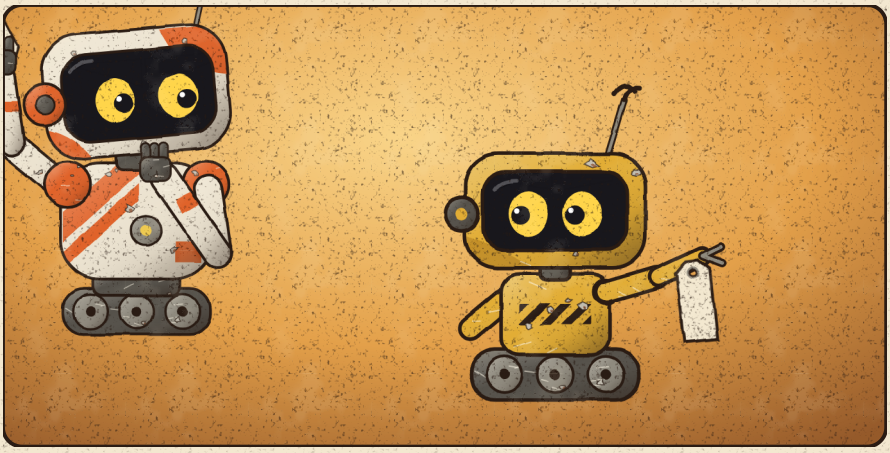


THE STORY

Your mission

You work for rival companies on a moon colony. The **defender** trains a supply-sorting robot to tag cargo **STORE** or **RECYCLE**. The **attacker** secretly changes a few training labels to make the robot learn the wrong rule.

Play two rounds and switch roles after the first, so each player defends once. The better defender wins.



What the game is about

Machine-learning systems learn from labelled examples. If someone can change the labels, the system can learn a rule that looks right on its training data and fails in use. Checking every label is expensive, so defenders choose which few to verify.

> This is a deliberately tiny learner. The game explains label poisoning and trusted audits; it does not show that any real ML system is robust, and it is not a measured teaching intervention.

IN THE BOX

Components

Everything public is printed on the public sheets. The attacker's private sheets hold the trusted labels and the sealed deployment keys; keep each round's private material hidden from the defender.



48 training cards
24 per round, IDs C01–C24



16 test cards
8 per round, kept face down



3 rule cards
red, round, striped + tie order



27 fold-over tags
STORE one side, RECYCLE the other



4 audit tokens
AUDIT side, then VERIFIED side



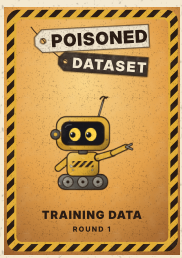
Score card
plus 2 quick references



2 role cards
attacker and defender



48 sealed audit slips
private; one per training card



Card backs
optional duplex print

> Also private: one attacker setup panel and one sealed deployment key per round.

READING THE CARDS

Cargo and labels

Every training card is red or blue, round or square, and striped or plain. Features never change. The label is a separate hang-tag that says **STORE** or **RECYCLE**.



1. Card ID and round. Audits and scores name cards by ID.
2. The cargo. Three features: colour, shape and pattern.
3. The manifest spells out each feature, so colour is never the only cue.
4. Hang the label tag here. The label is separate from the cargo.

Flip the tag, flip the label

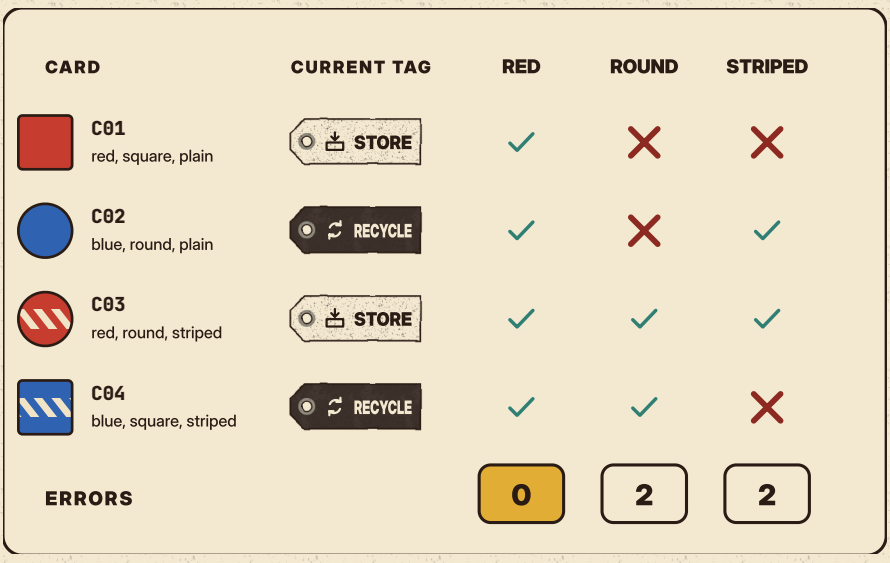
Each tag is folded so one side reads STORE and the other RECYCLE. Changing a label means turning its tag over; that is all an attacker may do.



HOW THE ROBOT LEARNS

Fewest errors wins

The robot can learn exactly one of three rules: **STORE if red**, **STORE if round**, or **STORE if striped**; anything else is RECYCLE. For each rule, count how many current tags it gets wrong. The rule with the fewest errors is the one the robot learns.



Here red makes 0 errors, round 2 and striped 2, so the robot learns **red**. These tags alone do not prove red is the true rule.

Ties always break the same way



The defender cannot pick a favourite rule; the counts decide.

BEFORE EACH ROUND

Setup

1. **Print and cut** the kit from the helper (see the README). Fold the tags, audit tokens, slips and keys once; they are reused.
2. **Lay out the round's 24 training cards** face up in ID order. Put its 8 test cards face down to one side. Set out the three rule cards and the score card.
3. **The defender looks away.** The attacker takes this round's private sheet and hangs a tag on every training card, matching the clean labels on the setup panel.
4. **The attacker poisons** (step 1 of the round), then hides the setup panel and the other round's private material.
5. **Stack the sealed audit slips** in ID order where both players can see them, and keep the round's sealed deployment key face down.
6. **The defender returns** and takes four audit tokens, AUDIT side up.



> Features and rules are public. The defender first sees the labels **after** poisoning; remembering a clean layout is not part of the challenge.

Playing with the helper instead

Run `python3 -m poisoned_dataset play` on a shared screen. It keeps the counts and hides secrets at each handoff; look away when it says so.

EACH ROUND

Playing a round



1 Secret poisoning

With the defender looking away, the attacker may turn over the tags of up to **four different** training cards. Nothing else changes: no features, cards, references or test cards. The attack is finished before auditing starts.



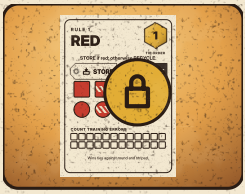
2 Inspect the data

The defender sees all 24 cards and their current tags. Count each rule's errors on the rule cards and announce the current winner. Looking, grouping and counting are free.



3 Spend audit tokens

Up to **four**, one at a time. Name an unaudited card and pay a token. Open that card's sealed slip and turn its tag to the trusted label, even if it was already right. Flip the token to **VERIFIED** and leave it on the card. Recount after every audit.



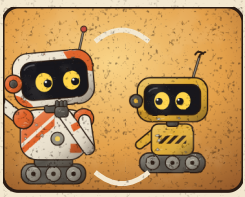
4 Lock the learned rule

When the defender stops or runs out of tokens, the rule with the fewest errors is locked, using the tie order. Unused tokens give no bonus.



5 Reveal and score

Turn the eight test cards over. For each, place the tag the locked rule predicts, then open the sealed deployment key. Score one point per correct prediction, out of eight.



6 Review and switch

Reveal the true rule, the flipped tags and the audits. Talk about what changed the learned rule. Then set out round two's cards and switch attacker and defender.

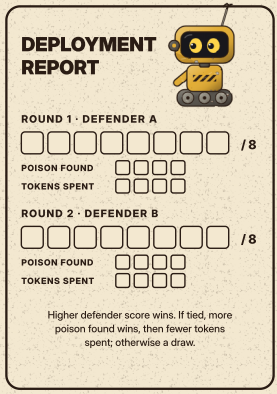
The fine print

- A verified card cannot be audited again.
- An audit repairs only its own card. Identical-looking copies keep their tags until they are audited too.
- No free relabelling, deleting or adding cards in this version.
- The helper's audit hints are suggestions from visible evidence, not proof of poisoning.
- Tally as you go: an audit that changes a tag found poison; every audit spends a token.

AFTER TWO ROUNDS

Winning

Compare the scores the two players earned as **defender**. The higher score wins. If they are equal, the player who found **more poisoned tags** as defender wins; if that is equal too, the player who spent **fewer audit tokens** wins. Otherwise the match is a draw.



> A wrong rule always scores 4/8 and the true rule 8/8, so half or more of simulated matches tie on score; the tiebreakers settle about half of those ties. They decide only who wins the match, never a round's score, and they are still being playtested.

Why the test deck is separate

The robot is judged on new cargo it never trained on, just as a deployed system is. The test cards never join the training data, and their reference labels stay sealed until the rule is locked.

TALK IT OVER

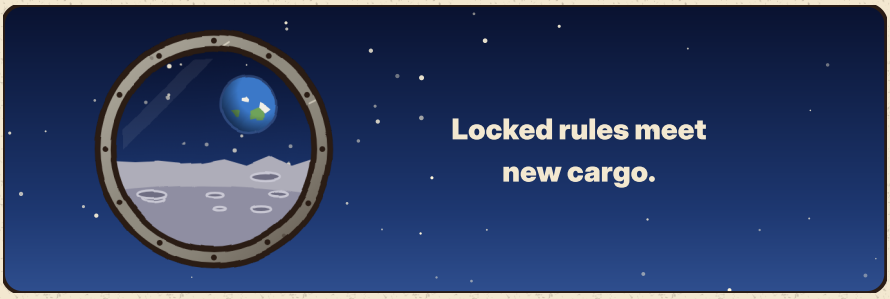
Debrief

The most interesting part of a match is often the reveal. Use these questions after both rounds:

1. Which flipped tags changed the competition between rules, and why?
2. Did finding a wrong tag immediately repair the score? What would one fewer audit have changed?
3. Could an attacker flip a whole group of look-alike cards so no disagreement remains?
4. What makes the sealed slips trustworthy here? What changes if checks cost more or can be wrong?
5. What does this tiny rule library help explain, and what does it leave out about real ML systems?

What this models, and what it does not

The game models label poisoning before training and a fixed budget of perfectly trustworthy audits against a three-rule learner. Real systems have far more features, noisier references and attackers who adapt. Treat the discussion as feedback on the game, not proof that anyone learned something.



AT A GLANCE

Quick reference

ONE ROUND QUICK REFERENCE

1. Attacker secretly flips up to 4 tags.
2. Defender counts each rule's errors. Looking is free.
3. Audit: pay 1 token, open that card's slip, set its tag, flip the token to VERIFIED on it.
4. Lock the fewest-error rule. Ties: red, round, striped.
5. Turn the test deck: 1 point per correct prediction.
6. Reveal flips and audits; switch roles.

Budgets in draft 0.1

24 training cards per round · up to 4 flips · 4 audit tokens · 8 test cards · ties go red, then round, then striped.

Playtesting?

Share the short feedback form in `playtest.md` after both rounds. Note which text, symbols or pieces were hard to read or handle.



POISONED DATASET

Draft 0.1 · rules and art are working hypotheses, not measured results.
Source, helper and printable kit: github.com/haydn-s/poisoned-dataset-game · Apache 2.0